Работа с камерой

Содержание лекции



Обзор API камеры в Android: Почему это сложно и какие есть пути?

Проблема: Почему работа с камерой — это сложно?

Фрагментация устройств: Тысячи моделей телефонов с разным «железом»: сенсоры, линзы, процессоры обработки изображений (ISP).

Особенности производителей (OEM): Каждый производитель (Samsung, Xiaomi, Google и др.) может вносить изменения в стандартное поведение камеры и реализацию API.

Разнообразие версий Android: API, разрешения и требования к безопасности (например, Scoped Storage) кардинально меняются от версии к версии.

Управление ресурсами: Камера — энергозатратный компонент. Неправильное управление жизненным циклом приводит к утечкам памяти, быстрой разрядке батареи и даже к системным сбоям.

Эволюция Camera API в Android

Camera API (до API 21) - deprecated

- Первый API. Простой в использовании для базовых задач, но имел множество багов и ограничений. Официально устарел.

Camera2 API (с API 21)

- Полная переработка. Дает мощный, низкоуровневый контроль над каждым аспектом камеры (экспозиция, фокус, ISO). Очень сложный, многословный и требует ручного управления состоянием и потоками.

CameraX (Jetpack)

- Современная абстракция над Camera2. Скрывает сложность, автоматически управляет жизненным циклом и решает проблемы совместимости. Рекомендованный Google подход.

Сравнение: Camera2 vs CameraX

Критерий	Camera2 API	CameraX
Объем кода	~300-500 строк для превью	~30-50 строк для превью
Жизненный цикл	Ручное управление в onResume, onPause	Автоматическое (Lifecycle-aware)
Совместимость	Ручная проверка и if-else для разных устройств	Встроена в библиотеку
Уровень абстракции	Низкоуровневый (потоки, состояния, сессии)	Высокоуровневый (сценарии использования)
Сложность	Очень высокая	Низкая / Средняя

Наш выбор — CameraX

CameraX — это официальная рекомендация от Google для разработки новых приложений.

Он является частью Android Jetpack — набора лучших практик и инструментов для современных Android-приложений.

Он не заменяет Camera2, а предоставляет удобный и безопасный фасад над ним.

Давайте погрузимся в его архитектуру и научимся им пользоваться!

CameraX: Архитектура и основы

Что такое CameraX? Ключевые принципы

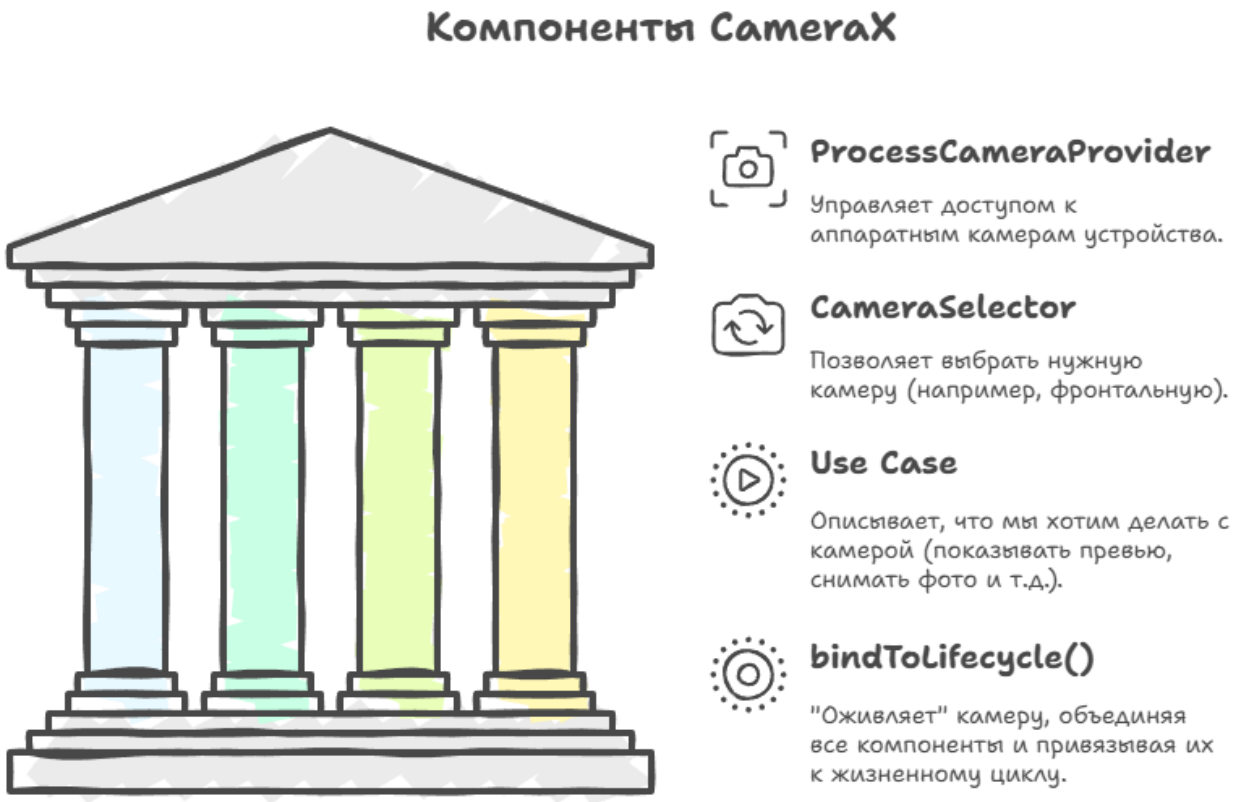
Часть Android Jetpack: Набор библиотек и инструментов от Google для следования лучшим практикам разработки.

Lifecycle-aware (Знает о жизненном цикле): Автоматически управляет ресурсами камеры (открытие, закрытие, пересоздание сессии) в ответ на события жизненного цикла (ON_START, ON_STOP).

Декларативный API: Вы описываете, что хотите получить (например, "превью и возможность делать фото"), а не реализуете как это сделать на низком уровне для каждого устройства.

Фасад над Camera2: Упрощает взаимодействие со сложным Camera2 API, не заменяя его.

Архитектура CameraX: Основные компоненты



Компонент 1: ProcessCameraProvider

Это синглтон, который управляет камерами в рамках всего приложения.

Его получение — асинхронная операция, так как инициализация камеры и запрос характеристик может занять время.

Компонент 1: ProcessCameraProvider

Получение экземпляра:

```
// Переменная для хранения Future
private lateinit var cameraProviderFuture:
ListenableFuture<ProcessCameraProvider>

// В onCreate() или onViewCreated()
cameraProviderFuture = ProcessCameraProvider.getInstance(requireContext())

// Добавляем Listener, который сработает, когда Provider будет готов
cameraProviderFuture.addListener({
    val cameraProvider = cameraProviderFuture.get()
    // ... здесь начинается настройка камеры
}, ContextCompat.getMainExecutor(requireContext()))
```

Компонент 2: CameraSelector

Отвечает за выбор физической камеры на основе заданных критериев.

Самые частые готовые селекторы:

- CameraSelector.DEFAULT_BACK_CAMERA — основная камера.
- CameraSelector.DEFAULT_FRONT_CAMERA — селфи-камера.

Для более сложных сценариев (например, выбор внешней USB-камеры) можно фильтровать камеры по их характеристикам с помощью addCameraFilter().

Компонент 2: CameraSelector

Создание собственного селектора через Builder:

```
val cameraSelector = CameraSelector.Builder()
    .requireLensFacing(CameraSelector.LENS_FACING_BACK) // или LENS_FACING_FRONT
    .build()
```

Компонент 3: Use Case

Use Case — это класс, который определяет конкретную задачу для камеры. Это основной «строительный блок» CameraX.

Мы детально разберем каждый из них в следующей части, а пока — краткий обзор:

- Preview: Отображение видеопотока на экране (видоискатель).
- ImageCapture: Съемка и сохранение фото.
- ImageAnalysis: Покадровый анализ изображения для компьютерного зрения.

Компонент 3: Use Case

Пример создания простого Preview Use Case:

```
val preview = Preview.Builder()
    // здесь могут быть настройки
    .build()
```

Связующее звено: bindToLifecycle()

Это главный метод CameraProvider, который «оживляет» камеру.

Что он делает:

1. Получает LifecycleOwner (например, this во фрагменте или viewLifecycleOwner).
2. Получает CameraSelector для выбора камеры.
3. Получает один или несколько созданных Use Case-ов.
4. Связывает их вместе: запускает камеру, когда жизненный цикл активен, и корректно освобождает ресурсы, когда он завершается.

Связующее звено: bindToLifecycle()

Сборка всего вместе (внутри listener-а):

```
try {
    // Отвязываем все от предыдущих использований
    cameraProvider.unbindAll()

    // Привязываем заново
    val camera = cameraProvider.bindToLifecycle(
        this, // LifecycleOwner
        cameraSelector,
        preview // ... и другие Use Cases
    )
} catch(exc: Exception) {
    Log.e(TAG, "Ошибка привязки UseCases", exc)
}
```

Сценарии использования (Use Cases)

Концепция Use Case

Use Case — это класс, реализующий одну конкретную задачу (сценарий) для камеры.

Это позволяет разделить логику работы с камерой на независимые, переиспользуемые блоки.

Четыре основных Use Case:

1. Preview: Отображение «живого» видеопотока на экране.
2. ImageCapture: Съемка и сохранение фото высокого качества.
3. ImageAnalysis: Доступ к каждому кадру для анализа (CV, ML).
4. VideoCapture: Запись видео и аудио.

Сценарии можно комбинировать, привязывая несколько Use Case к одному жизненному циклу одновременно.

Use Case: Preview

Задача: Отобразить «видоискатель» на экране, то, что видит камера в реальном времени.

Как работает: Preview Use Case генерирует видеопоток, который затем нужно направить на специальную View для отображения.

Компонент для отображения: androidx.camera.view.PreviewView.

- Это View, специально созданная для CameraX.
- Она автоматически обрабатывает повороты, масштабирование и соотношение сторон, чтобы изображение не искажалось.

Preview: Шаг 1 — PreviewView в XML-разметке

Первый шаг — добавить PreviewView в ваш layout-файл.

Обычно он занимает весь экран или его значительную часть.

```
<!-- res/layout/fragment_camera.xml -->
<androidx.constraintlayout.widget.ConstraintLayout ...>

    <androidx.camera.view.PreviewView
        android:id="@+id/previewView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button
        android:id="@+id/captureButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Сделать фото"
        ... />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Preview: Шаг 2 — Настройка и привязка

В коде, внутри listener-а CameraProvider, мы создаем Preview и связываем его с PreviewView.

```
// (внутри addListener после получения cameraProvider)

// 1. Создаем Preview Use Case
val preview = Preview.Builder().build()

// 2. Выбираем заднюю камеру
val cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA
```

Preview: Шаг 2 — Настройка и привязка

setSurfaceProvider — это ключевой метод, который "передает" поток кадров из Preview в PreviewView.

```
// 3. Привязываем Use Case к жизненному циклу
// (перед этим не забываем вызвать cameraProvider.unbindAll())
cameraProvider.bindToLifecycle(this, cameraSelector, preview)

// 4. Связываем наш PreviewView из XML с Use Case
preview.setSurfaceProvider(binding.previewView.surfaceProvider)
```

Use Case: ImageCapture

Задача: Делать качественные фотографии и сохранять их.

Этот Use Case предоставляет:

- Простой метод takePicture() для съемки.
- Автоматические функции 3A: Auto-Focus, Auto-Exposure, Auto-White-Balance.
- Возможность управлять вспышкой.
- Настройку оптимизации: по качеству или по скорости съемки.

ImageCapture: Настройка и добавление в bindToLifecycle

Мы создаем экземпляр ImageCapture и просто добавляем его в тот же вызов bindToLifecycle, что и Preview.

```
// Объявляем переменную на уровне класса, чтобы иметь к ней доступ
private var imageCapture: ImageCapture? = null

// ...
// (внутри addListener)

val preview = Preview.Builder().build()

// 1. Создаем ImageCapture с настройками
imageCapture = ImageCapture.Builder()
    // Можно установить режим оптимизации
    // .setCaptureMode(ImageCapture.CAPTURE_MODE_MAXIMIZE_QUALITY)
    .build()
```

ImageCapture: Настройка и добавление в bindToLifecycle

Мы создаем экземпляр ImageCapture и просто добавляем его в тот же вызов bindToLifecycle, что и Preview.

```
// 2. Добавляем его в список Use Cases при привязке
cameraProvider.bindToLifecycle(
    this,
    cameraSelector,
    preview,
    imageCapture // <-- ДОБАВИЛИ СЮДА
)

preview.setSurfaceProvider(binding.previewView.surfaceProvider)
```

ImageCapture: Съемка фото с помощью takePicture()

Метод takePicture() вызывается по действию пользователя (например, по клику на кнопку). Основной сценарий — сохранение в файл:

```
// Слушатель для кнопки
binding.captureButton.setOnClickListener {
    // Получаем стабильную ссылку на use case
    val imageCapture = imageCapture ?: return@setOnClickListener

    // 1. Создаем файл для сохранения
    val photoFile = File(outputDirectory,
        SimpleDateFormat(FILENAME_FORMAT, Locale.US)
            .format(System.currentTimeMillis()) + ".jpg")
    …

}
```

ImageCapture: Съемка фото с помощью takePicture()

Метод takePicture() вызывается по действию пользователя (например, по клику на кнопку). Основной сценарий — сохранение в файл:

```
// Слушатель для кнопки
binding.captureButton.setOnClickListener {
    …
    // 2. Создаем опции вывода, указывая файл
    val outputOptions = ImageCapture.OutputFileOptions.Builder(photoFile).build()
    …
}
```

ImageCapture: Съемка фото с помощью takePicture()

```
// Слушатель для кнопки
binding.captureButton.setOnClickListener {
    …
    // 3. Вызываем takePicture
    imageCapture.takePicture(
        outputOptions,
        ContextCompat.getMainExecutor(requireContext()),
        object : ImageCapture.OnImageSavedCallback {
            override fun onImageSaved(output: OutputFileResults) {
                // Файл успешно сохранен
            }
            override fun onError(exc: ImageCaptureException) {
                // Ошибка при съемке или сохранении
            }
        })
}
```

Use Case: ImageAnalysis

Задача: Предоставить доступ к каждому кадру с камеры для анализа без отображения на экране.

Приложение получает буфер с данными изображения, доступный для CPU.

Примеры использования:

- Сканирование QR-кодов и штрих-кодов.
- Распознавание лиц (детекция, эмоции).
- Распознавание текста (OCR).
- Применение фильтров/эффектов в реальном времени (хотя для этого лучше GPU).
- Любая задача компьютерного зрения (CV) или машинного обучения (ML).

ImageAnalysis: Реализация Analyzer

Для обработки кадров нужно создать класс, реализующий интерфейс ImageAnalysis.Analyzer.

У него всего один метод — analyze(), который вызывается для каждого кадра.

Ключевое правило: Обязательно вызывать imageProxy.close() в конце! Иначе новые кадры перестанут поступать.

ImageAnalysis: Реализация Analyzer

```
private class MyImageAnalyzer : ImageAnalysis.Analyzer {
    override fun analyze(imageProxy: ImageProxy) {
        // Здесь происходит вся магия анализа
        // Например, можно получить формат и размеры
        val format = imageProxy.format
        val rotationDegrees = imageProxy.imageInfo.rotationDegrees

        // ... код для распознавания QR-кода ...

        // ВАЖНО: Закрываем imageProxy, чтобы освободить буфер
        // и получить следующий кадр.
        imageProxy.close()
    }
}
```

ImageAnalysis: Настройка и привязка

ImageAnalysis создается через Builder и привязывается к жизненному циклу вместе с другими Use Cases.

```
// (внутри addListener)

// 1. Создаем ImageAnalysis
val imageAnalysis = ImageAnalysis.Builder()
    // Устанавливаем стратегию обработки кадров
    // STRATEGY_KEEP_ONLY_LATEST: если анализ тормозит, старые кадры
отбрасываются
    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
    .build()

…
```

ImageAnalysis: Настройка и привязка

ImageAnalysis создается через Builder и привязывается к жизненному циклу вместе с другими Use Cases.

```
// (внутри addListener)

…
// 2. Устанавливаем наш анализатор
imageAnalysis.setAnalyzer(
    Executors.newSingleThreadExecutor(), // Анализ будет идти в отдельном потоке
    MyImageAnalyzer()
)

…
```

ImageAnalysis: Настройка и привязка

ImageAnalysis создается через Builder и привязывается к жизненному циклу вместе с другими Use Cases.

```
// (внутри addListener)

…

// 3. Добавляем в bindToLifecycle
cameraProvider.bindToLifecycle(
    this,
    cameraSelector,
    preview,
    imageCapture,
    imageAnalysis // <-- ДОБАВИЛИ СЮДА
)
```

Use Case: VideoCapture

Задача: Запись видео и аудио.

Современная версия VideoCapture (VideoCapture 2.0) была значительно переработана и стабилизирована.

Ключевые возможности:

- Простое API для старта и остановки записи.
- Запись в File, FileDescriptor или MediaStore.
- Позволяет получать статистику записи (размер файла, длительность).
- Работает в комбинации с Preview.

VideoCapture: Настройка и привязка

VideoCapture требует Recorder для конфигурации качества и executor.
Привязывается к жизненному циклу аналогично другим Use Case-ам.

```
// Переменная на уровне класса
private var videoCapture: VideoCapture<Recorder>? = null

…
```

VideoCapture: Настройка и привязка

VideoCapture требует Recorder для конфигурации качества и executor. Привязывается к жизненному циклу аналогично другим Use Case-ам.

```
// ...
// (внутри addListener)

val preview = Preview.Builder().build()

// 1. Создаем Recorder с настройками качества
val recorder = Recorder.Builder()
    .setQualitySelector(QualitySelector.from(Quality.HIGHEST))
    .build()
…
```

VideoCapture: Настройка и привязка

VideoCapture требует Recorder для конфигурации качества и executor. Привязывается к жизненному циклу аналогично другим Use Case-ам.

```
// ...
// (внутри addListener)

// 2. Создаем VideoCapture на основе Recorder'a
videoCapture = VideoCapture.withOutput(recorder)

…
```

VideoCapture: Настройка и привязка

VideoCapture требует Recorder для конфигурации качества и executor.
Привязывается к жизненному циклу аналогично другим Use Case-ам.

```
// ...
// (внутри addListener)

// 3. Добавляем его в список Use Cases при привязке
cameraProvider.bindToLifecycle(
    this,
    cameraSelector,
    preview,
    videoCapture // <-- ДОБАВИЛИ СЮДА
)

preview.setSurfaceProvider(binding.previewView.surfaceProvider)
```

VideoCapture: Старт и остановка записи

Запись управляется методами start и stop. Процесс записи отслеживается через колбэки.
Для записи требуется разрешение RECORD_AUDIO.

```
// Переменная для отслеживания текущей записи
private var currentRecording: Recording? = null

…
```

VideoCapture: Старт и остановка записи

```
…
// По клику на кнопку "Запись"
private fun startRecording() {
    // 1. Настраиваем, куда сохранять видео (в MediaStore)
    val name = "CameraX-video-" + SimpleDateFormat(FILENAME_FORMAT, Locale.US)
        .format(System.currentTimeMillis()) + ".mp4"
    val contentValues = ContentValues().apply { ... }
    val mediaStoreOutput = MediaStoreOutputOptions
        .Builder(contentResolver, MediaStore.Video.Media.EXTERNAL_CONTENT_URI)
        .setContentValues(contentValues)
        .build()
    …
}
…
```

VideoCapture: Старт и остановка записи

```
…
// По клику на кнопку "Запись"
private fun startRecording() {
    …
    // 2. Начинаем запись
    currentRecording = videoCapture.output
        .prepareRecording(this, mediaStoreOutput)
        .withAudioEnabled() // Не забываем запросить разрешение
        .start(ContextCompat.getMainExecutor(this)) { recordEvent ->
            when(recordEvent) {
                is VideoRecordEvent.Start -> { /* Запись началась */ }
                is VideoRecordEvent.Finalize -> { /* Запись завершена */ }
            }
        }
}
…
```

VideoCapture: Старт и остановка записи

```
// Переменная для отслеживания текущей записи
private var currentRecording: Recording? = null

// По клику на кнопку "Запись"
private fun startRecording() {
    …
}

// По клику на кнопку "Стоп"
currentRecording?.stop()
```

Продвинутое управление: Зум, фокус, разрешения и другие тонкости.

Выход за рамки Use Cases: CameraControl и CameraInfo

После успешной привязки bindToLifecycle() возвращает объект Camera.

Этот объект — наш шлюз к динамическому управлению и получению информации.

- camera.cameraControl: Позволяет изменять параметры камеры «на лету» (включить вспышку, изменить зум).
- camera.cameraInfo: Позволяет получать информацию о текущем состоянии камеры (поддерживается ли вспышка, текущий уровень зума, ориентация сенсора).

Выход за рамки Use Cases: CameraControl и CameraInfo

```
// (внутри bindToLifecycle)
val camera = cameraProvider.bindToLifecycle(...)

// Получаем доступ к контроллерам
val cameraControl = camera.cameraControl
val cameraInfo = camera.cameraInfo
```

Управление зумом и вспышкой

Зум (Zoom):

- cameraInfo.zoomState (LiveData) — позволяет получить информацию о текущем и максимальном/минимальном уровне зума.

- cameraControl.setZoomRatio(ratio) — устанавливает уровень зума. ratio - это float от minZoomRatio до maxZoomRatio.

Вспышка/Фонарик (Flash/Torch):

- cameraInfo.hasFlashUnit() — проверяет, есть ли на устройстве вспышка.

- cameraControl.enableTorch(true/false) — включает/выключает фонарик.

- Для ImageCapture режим вспышки (FLASH_MODE_ON/AUTO/OFF) задается в ImageCapture.Builder или imageCapture.flashMode.

Управление зумом и вспышкой

```
// Пример управления фонариком
if (cameraInfo.hasFlashUnit()) {
    cameraControl.enableTorch(true) // Включить
}
```

Tap-to-Focus (Фокусировка по касанию)

Реализуется через CameraControl.startFocusAndMetering().

Алгоритм:

1. На PreviewView вешается слушатель касаний (OnTouchEvent).
2. Из события касания получаем координаты (x, y).
3. Создаем MeteringPoint на основе этих координат.
4. Создаем FocusMeteringAction на основе точки.
5. Вызываем startFocusAndMetering().

Tap-to-Focus (Фокусировка по касанию)

```
binding.previewView.setOnTouchListener { _, event ->
    // 1. Создаем точку из координат касания
    val meteringPoint = binding.previewView.meteringPointFactory
        .createPoint(event.x, event.y)
    // 2. Создаем действие
    val action = FocusMeteringAction.Builder(meteringPoint).build()
    // 3. Запускаем фокусировку
    cameraControl.startFocusAndMetering(action)
    true
}
```

Управление разрешением и соотношением сторон

По умолчанию CameraX сам выбирает оптимальное разрешение, чтобы избежать сбоев. Но им можно управлять.

Два способа (в Builder-е Use Case):

- setTargetAspectRatio(AspectRatio.RATIO_16_9): Задать соотношение сторон (4:3, 16:9). CameraX подберет **наилучшее доступное** разрешение с этим соотношением. **Рекомендуемый способ.**

- setTargetResolution(Size(1280, 720)): Задать конкретное разрешение. CameraX попытается найти **ближайшее подходящее**. Полезно для ML-моделей, требующих фиксированного входа.

Важно: Нельзя вызывать setTargetResolution и setTargetAspectRatio одновременно. Это приведет к ошибке.

Комбинации Use Cases и аппаратные уровни

Не все устройства могут поддерживать сложные комбинации Use Case-ов одновременно.

Возможности камеры определяются ее аппаратным уровнем (Hardware Level):

- LEGACY: Старые устройства. Ограниченные возможности.
- LIMITED: Поддерживает часть возможностей Camera2.
- FULL: Поддерживает все основные возможности Camera2.
- LEVEL_3: Устройства топ-уровня. Поддержка YUV-репроцессинга, RAW-съемки.

Пример ограничения: Preview + VideoCapture + ImageAnalysis одновременно могут требовать уровня LEVEL_3.

Если комбинация не поддерживается, bindToLifecycle() выбросит исключение.

ViewPort и кадрирование (Cropping)

Проблема: Разрешение сенсора камеры, Preview, и ImageCapture могут иметь разное соотношение сторон. Как гарантировать, что пользователь сфотографирует именно то, что видит в видоискателе?

Решение: ViewPort и UseCaseGroup.

ViewPort описывает видимую пользователю область (например, PreviewView).

UseCaseGroup объединяет несколько Use Case-ов и применяет к ним общий ViewPort.

CameraX автоматически кадрирует выход всех Use Case-ов в группе так, чтобы они соответствовали одной и той же области на сенсоре. Это продвинутая, но очень полезная фича для достижения эффекта WYSIWYG — «что видишь, то и получаешь».

ViewPort и кадрирование (Cropping)

```
val viewPort = binding.previewView.viewPort!!
val useCaseGroup = UseCaseGroup.Builder()
    .setViewPort(viewPort)
    .addUseCase(preview)
    .addUseCase(imageCapture)
    .build()
cameraProvider.bindToLifecycle(this, cameraSelector, useCaseGroup)
```

Настройки PreviewView: ImplementationMode и ScaleType

PreviewView имеет две важные настройки для оптимизации и UI.
implementationMode:

- PERFORMANCE (по умолчанию): Использует SurfaceView. Максимальная производительность, минимальные задержки. Но сложно накладывать другие View поверх.
- COMPATIBLE: Использует TextureView. Позволяет легко делать анимации и трансформации (прозрачность, вращение), но может влиять на производительность.

scaleType:

- FILL_CENTER (по умолчанию): Изображение заполняет весь View, обрезая лишнее.
- FIT_CENTER: Изображение вписывается в View целиком, по бокам могут быть черные полосы (letterboxing).

PreviewView: SurfaceView vs TextureView

PreviewView может использовать два разных типа «поверхностей» для отрисовки видеопотока. Это управляется через implementationMode.

SurfaceView (Режим PERFORMANCE)

- Как работает: Создает отдельное «окно» (surface), которое находится под окном вашего приложения. Композитор Android просто «прорубает» дыру в вашем UI, чтобы показать его.

- Плюсы: ✅ Максимальная производительность. Минимальное энергопотребление. Нулевое копирование видео-буфера.

- Минусы: ❌ Нельзя применять трансформации (alpha, rotation). Сложно размещать другие View поверх (проблемы с Z-order на старых API).

PreviewView: SurfaceView vs TextureView

PreviewView может использовать два разных типа «поверхностей» для отрисовки видеопотока. Это управляется через implementationMode.

TextureView (Режим COMPATIBLE)

- Как работает: Ведет себя как обычное View. Видеопоток отрисовывается в текстуру, которая затем компонуется вместе с остальным UI.
- Плюсы: ✅ Полная поддержка всех трансформаций и анимаций. Легко вписывается в любой layout.
- Минусы: ❌ Менее производительный. Потребляет больше памяти и энергии.

Настройка ScaleType: FIT vs FILL

ScaleType определяет, как изображение с камеры (source) вписывается в PreviewView (destination).

FIT_* (вписать): FIT_CENTER, FIT_START, FIT_END

- Цель: Показать весь кадр без обрезки.
- Результат: Соотношение сторон сохраняется. Если оно не совпадает с PreviewView, по бокам появятся пустые полосы (Letterboxing).
- Использование: Когда важно видеть всё, что попадает в объектив (например, камеры наблюдения).

Настройка ScaleType: FIT vs FILL

ScaleType определяет, как изображение с камеры (source) вписывается в PreviewView (destination).

FILL_* (заполнить): FILL_CENTER, FILL_START, FILL_END

- Цель: Заполнить весь PreviewView без пустых полос.
- Результат: Соотношение сторон сохраняется. Если оно не совпадает, часть кадра будет обрезана (Cropping).
- Использование: Самый частый случай. Создает эффект «полноэкранной» камеры, что более привычно пользователям. FILL_CENTER является режимом по умолчанию.

Процесс масштабирования: Пример с FIT_CENTER

Дано:
Исходное видео (Source): 640x480 (соотношение 4:3)
PreviewView (Destination): 1080x1920 (соотношение 9:16)



Процесс масштабирования: Пример с FIT_CENTER

Шаг 1: Вычислить коэффициент масштабирования.

- FIT использует min(dst/src).
- scaleX = 1080 / 640 = 1.6875
- scaleY = 1920 / 480 = 4.0
- scale = min(1.6875, 4.0) = 1.6875

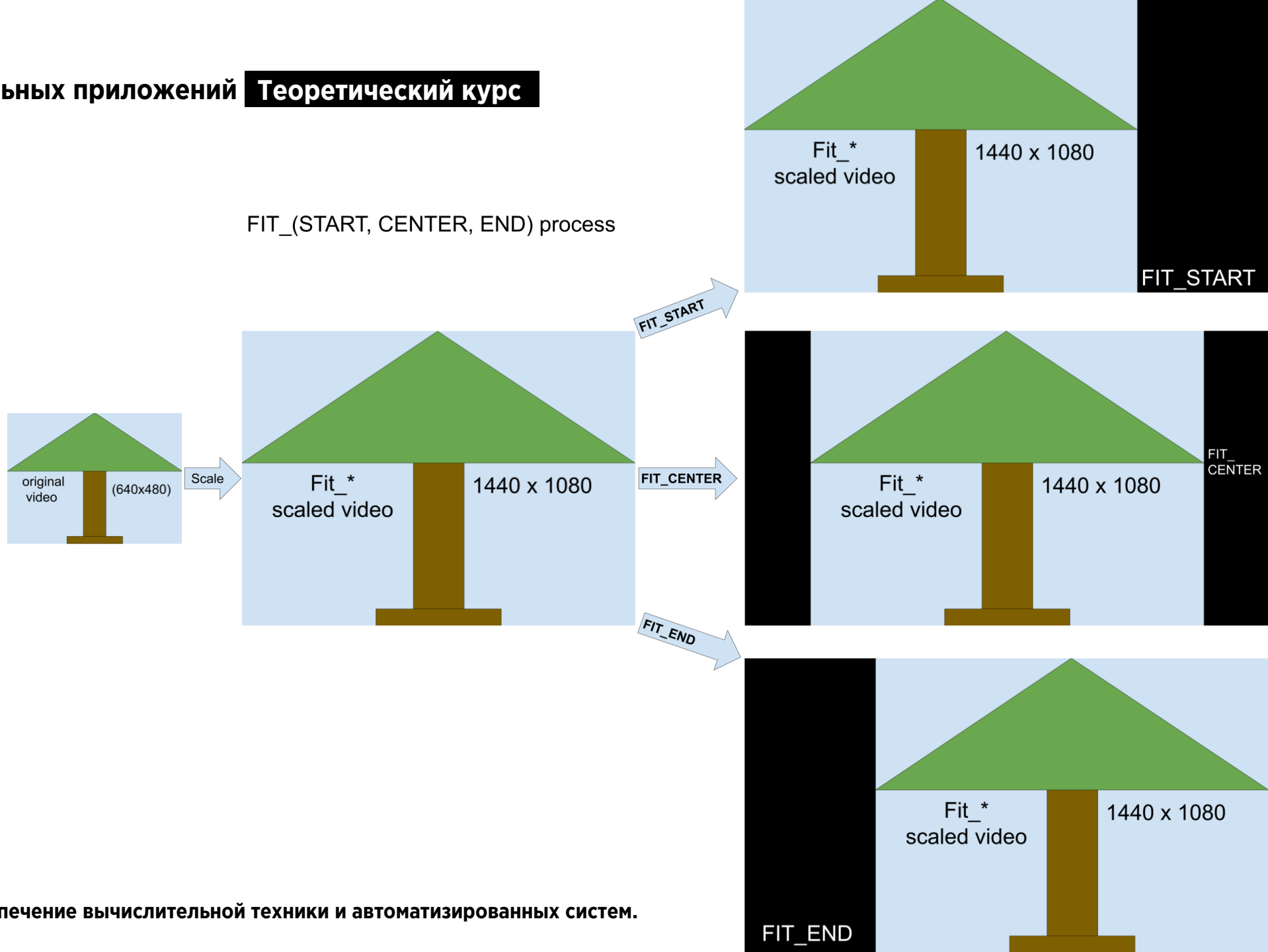
Шаг 2: Применить масштабирование к исходному видео.

- newWidth = 640 * 1.6875 = 1080
- newHeight = 480 * 1.6875 = 810
- Итог: видео размером 1080x810.

Шаг 3: Разместить в PreviewView (1080x1920).

- FIT_CENTER размещает видео по центру.
- По горизонтали совпадает. По вертикали остается (1920 - 810) / 2 = 555 пикселей пустого пространства сверху и снизу.

FIT_(START, CENTER, END) process



Процесс масштабирования: Пример с FIT_CENTER

Шаг 1: Вычислить коэффициент масштабирования.

- FILL использует max(dst/src).
- scaleX = 1080 / 640 = 1.6875
- scaleY = 1920 / 480 = 4.0
- scale = max(1.6875, 4.0) = 4.0

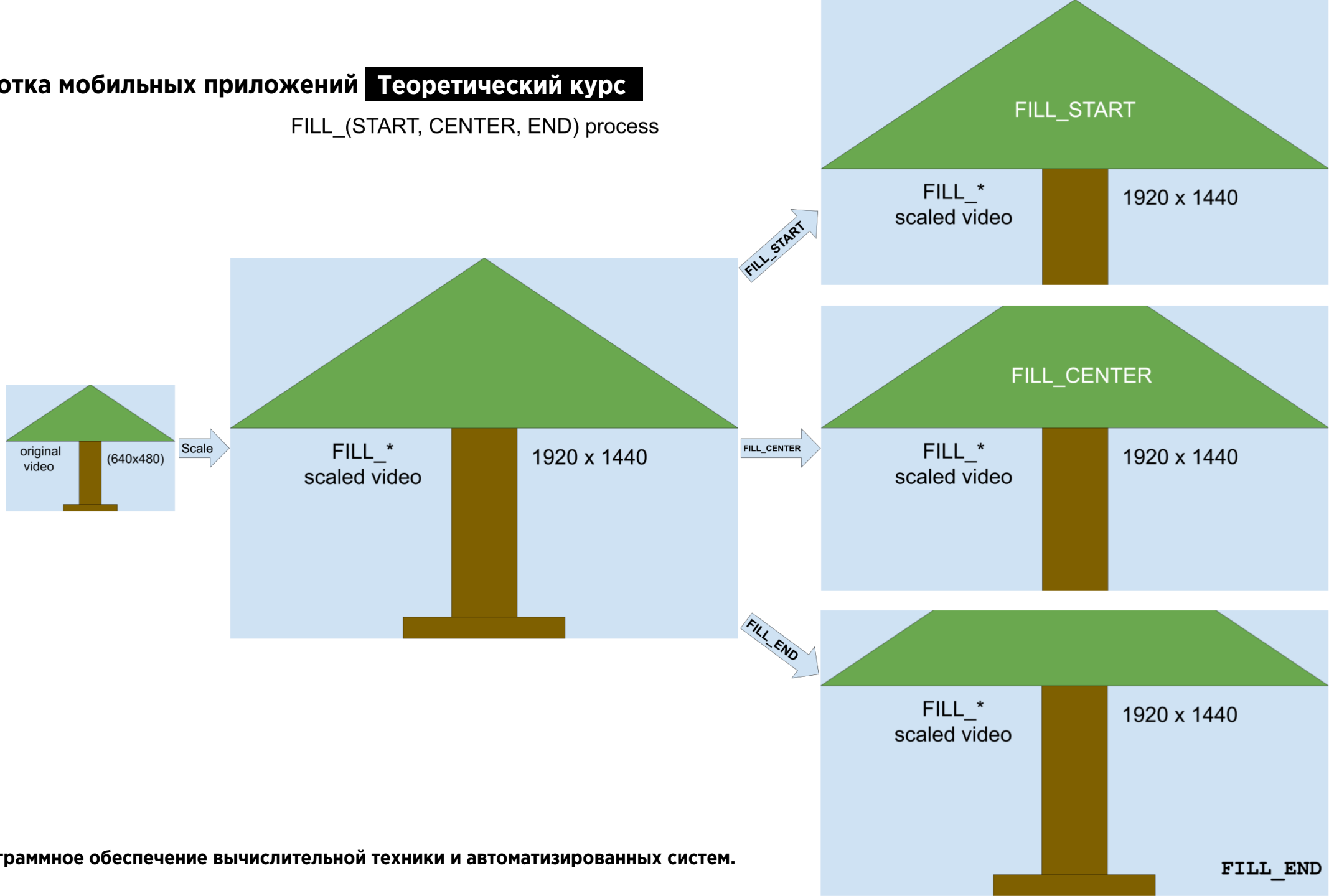
Шаг 2: Применить масштабирование к исходному видео.

- newWidth = 640 * 4.0 = 2560
- newHeight = 480 * 4.0 = 1920
- Итог: видео размером 2560x1920.

Шаг 3: Обрезать до размеров PreviewView (1080x1920).

- FILL_CENTER центрирует и обрезает.
- По высоте совпадает. По горизонтали обрезается (2560 - 1080) / 2 = 740 пикселей слева и справа.

FILL_(START, CENTER, END) process



Управление потоками (Threading)

Проблема: Многие операции с камерой (инициализация, анализ кадров) являются «тяжелыми» и могут заблокировать главный поток (UI Thread), вызвав зависание приложения (ANR).

Решение CameraX: По умолчанию CameraX выполняет всю тяжелую работу в своих внутренних фоновых потоках.

Когда нужен ручной контроль?

- Если у вас в приложении уже есть своя архитектура потоков (например, пул потоков для обработки данных).
- Для выполнения тестов, требующих синхронизации.
- Для строгой оптимизации ресурсов.

Управление потоками (Threading)

Настройка:

- Через CameraXConfig (при инициализации приложения).
- При настройке Use Case, например, ImageAnalysis.setAnalyzer(executor, ...).

```
// Пример установки собственного Executor для ImageAnalysis
imageAnalysis.setAnalyzer(
    Executors.newSingleThreadExecutor(), // Создаем выделенный поток
    MyImageAnalyzer()
)
```

Лучшие практики и альтернативы: Когда использовать Intent и как писать надежный код.

Альтернативный путь: когда использовать простой Intent?

Не всегда нужно использовать CameraX. Для самых простых задач есть более легкий путь.

Используйте Intent(MediaStore.ACTION_IMAGE_CAPTURE) если:

- ✅ Простота важнее контроля. Вам нужно получить фото «здесь и сейчас», с минимальным количеством кода.
- ✅ Не важен UI. Вас полностью устраивает интерфейс системного приложения камеры.
- ✅ Нужна одна фотография. Вам не нужен «режим камеры» внутри вашего приложения.

Альтернативный путь: когда использовать простой Intent?

Не всегда нужно использовать CameraX. Для самых простых задач есть более легкий путь.

Не используйте Intent если:

- ❌ Вам нужен кастомный UI/UX. (Свои кнопки, фильтры, оверлеи).
- ❌ Вам нужен доступ к видеопотоку. (Для превью или анализа).
- ❌ Вам нужна высокая производительность или серийная съемка.
- ❌ Вам нужен полный контроль над параметрами съемки.

Рецепт: Intent + FileProvider для сохранения фото

Шаг 1: AndroidManifest.xml — Декларация FileProvider.

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider" ... >
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

Рецепт: Intent + FileProvider для сохранения фото

Шаг 2: res/xml/file_paths.xml — Определение путей.

```
<paths>
    <cache-path name="images" path="images/"/>
</paths>
```

Рецепт: Intent + FileProvider для сохранения фото

Шаг 3: Код — Создание файла, URI и запуск Intent.

```
val file = File(context.cacheDir, "images/my_photo.jpg")
val uri = FileProvider.getUriForFile(context,
"${context.packageName}.fileprovider", file)
val intent =
Intent(MediaStore.ACTION_IMAGE_CAPTURE).putExtra(MediaStore.EXTRA_OUTPUT, uri)
// Запуск через ActivityResultLauncher
```

Практика №1: Правильный запрос разрешений

Работа с камерой требует разрешений. Их нужно запрашивать в рантайме.
Необходимые разрешения в AndroidManifest.xml:

```
<uses-feature android:name="android.hardware.camera.any" />
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.RECORD_AUDIO" /> <!-- Для видео -->
<!-- WRITE_EXTERNAL_STORAGE может не требоваться для Android 10+ -->
```

Практика №1: Правильный запрос разрешений

Современный подход — ActivityResultLauncher:

```
private val requestPermissionLauncher =
    registerForActivityResult(ActivityResultContracts.RequestPermission()) { isGranted ->
        if (isGranted) {
            // Разрешение получено, запускаем камеру
            startCamera()
        } else {
            // Показываем объяснение или отключаем функцию
        }
    }

// В коде, перед запуском камеры
requestPermissionLauncher.launch(Manifest.permission.CAMERA)
```

Практика №2: Жизненный цикл и потокобезопасность

Всегда используйте viewLifecycleOwner во фрагментах.

- Жизненный цикл View фрагмента отличается от жизненного цикла самого фрагмента. Привязка к viewLifecycleOwner гарантирует, что камера освободится, даже если фрагмент остался в back stack.

Вызывайте cameraProvider.unbindAll() перед bindToLifecycle().

- Это предотвращает конфликты, если Use Case-ы уже были привязаны ранее.

Доступ к CameraControl и ImageCapture должен быть потокобезопасным.

- Сохраняйте их в переменные класса и проверяйте на null перед использованием, особенно если конфигурация может измениться.

Выполняйте «тяжелый» анализ в фоновом потоке.

- Используйте Executor в ImageAnalysis.setAnalyzer(), чтобы не блокировать UI.

Практика №3: Тестирование

Эмулятор — это только начало.

- Эмулятор отлично подходит для отладки логики, но не может симулировать все особенности реального "железа".

Тестируйте на реальных устройствах.

- Разные производители: Проверьте на условных Samsung, Xiaomi, Google Pixel. Вы можете столкнуться с OEM-специфичными багами.

- Разные версии Android: Особенно проверьте поведение на границах API (например, Android 10/11 с их Scoped Storage).

- «Слабые» устройства: Проверьте производительность на бюджетных моделях. Убедитесь, что ImageAnalysis не приводит к зависаниям.

Практика №3: Тестирование

Тестируйте крайние случаи:

- Что происходит при быстром повороте экрана?
- Что если пользователь сворачивает и разворачивает приложение во время записи видео?
- Что если разрешения отозваны во время работы приложения?